

Data Lab
Mathematical Institute for Machine Learning and Data Science
of the Katholische Universität Eichstätt-Ingolstadt

Comparison of Feature Selection Methods

Examiner: Prof. Dr. Tijana Janjic
Heisenberg Professorship for Data Assimilation
Katholische Universität Eichstätt-Ingolstadt

Advisor: Dr. Maryam Ramezani Ziarani

Submitted by:	Manuel Cortés	Matriculation Number: 11017929
	Course of Study:	Bachelor in Data Science
	Elmir Dashtiyev	Matriculation Number: 11021397
	Course of Study:	Bachelor in Data Science

Submitted on: August 14, 2025

Contents

1	Introduction	1
2	Materials and Methods	5
2.1	Dataset	5
2.2	Feature Selection with Information Imbalance Method	7
2.3	Code Implementation	7
2.4	Workflow Overview	9
3	Results	11
3.1	Feature Selection with the Information Imbalance Method	11
3.2	Comparing Top Features vs. All Features	12
3.3	Feature Selection with Neural Network Backpropagation	13
3.4	Summary of Observations	14
4	Conclusion	15
4.1	Overall Findings	15
4.2	Recommendations	15
4.3	Comparison Figures	16
	Bibliography	18

1 Introduction

Feature selection is a key step in making machine learning (ML) models both easier to understand and more efficient. This is especially important when we have a large number of features but only a small number of data samples, which is often the case in fields like physics or when modeling complex systems. In 2023, Glielmo et al. introduced a new method that helps compare how much useful information different sets of features actually carry. What makes their approach stand out is that it works well even when the data is nonlinear or lives in a high-dimensional space. It is a helpful tool for figuring out which features are truly important.

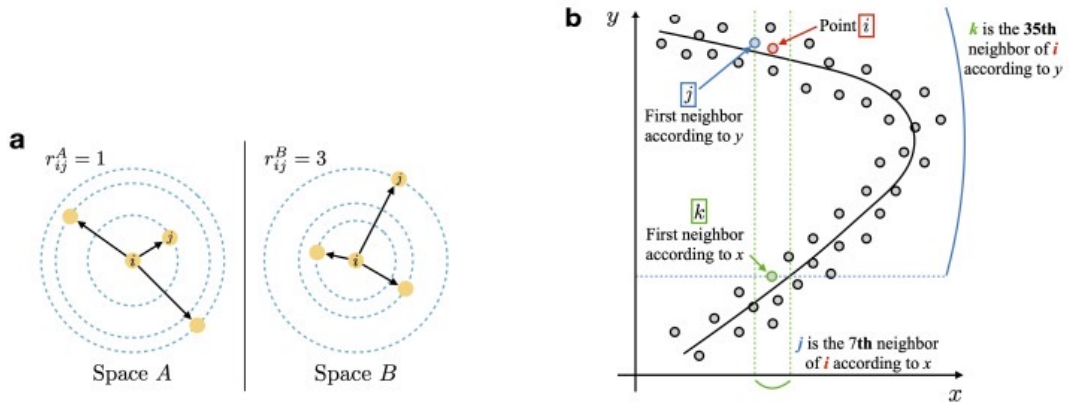


Figure 1: A point can be a close neighbor in one feature space but not in another, showing that one space may contain more predictive information. If knowing neighbors in space Y helps predict those in X (but not vice versa), Y is more informative.

The main idea in the paper is something called *information imbalance*. This concept helps determine whether one set of features (or the way we measure distance between data points using those features) carries more useful information than another. The approach is intuitive: if the nearest neighbor of a data point in feature space A is also close in feature space B, then A likely “knows” what B knows. But if that neighbor

looks far away in B, the two feature spaces are probably capturing different aspects of the data.

To measure this, the method compares how neighbor ranks change between feature spaces, and uses a copula-based transformation to make sure the results are not affected by scaling or odd data distributions. The final score, called information imbalance $\Delta(A \rightarrow B)$, tells us how informative A is about B. A score of 0 means A fully explains B, while 1 means A tells us nothing about B. This asymmetry is especially useful for selecting the most important features and can even hint at causal relationships.

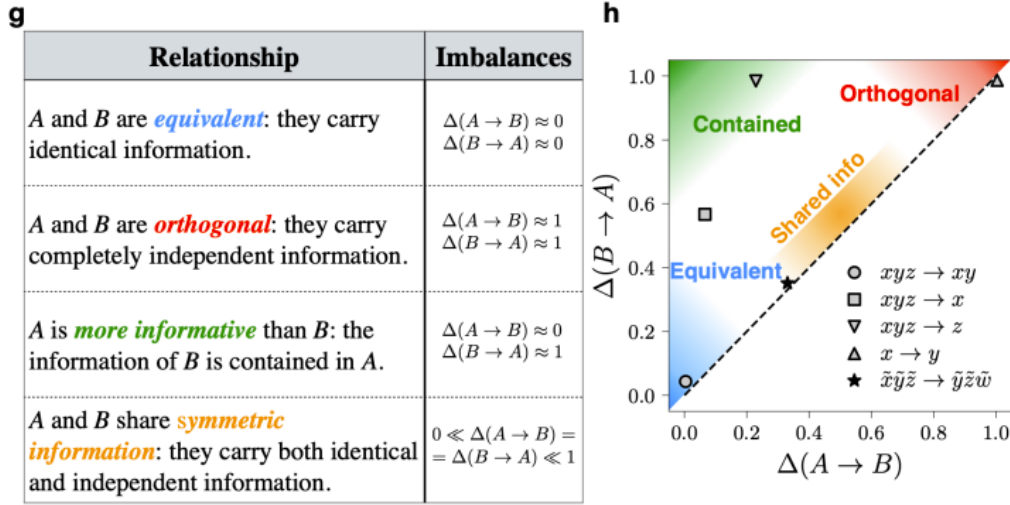


Figure 2: (g) Categories of relationships between two feature spaces. (h) Visualization on synthetic data using the information imbalance plane, showing differences in informativeness.

In machine learning and physical modeling, the way we measure distances between data points usually comes from the features we use. Glielmo and his team look at feature selection in a new way: they reinterpret it as the task of finding the most informative set of features—or equivalently, the most informative distances.

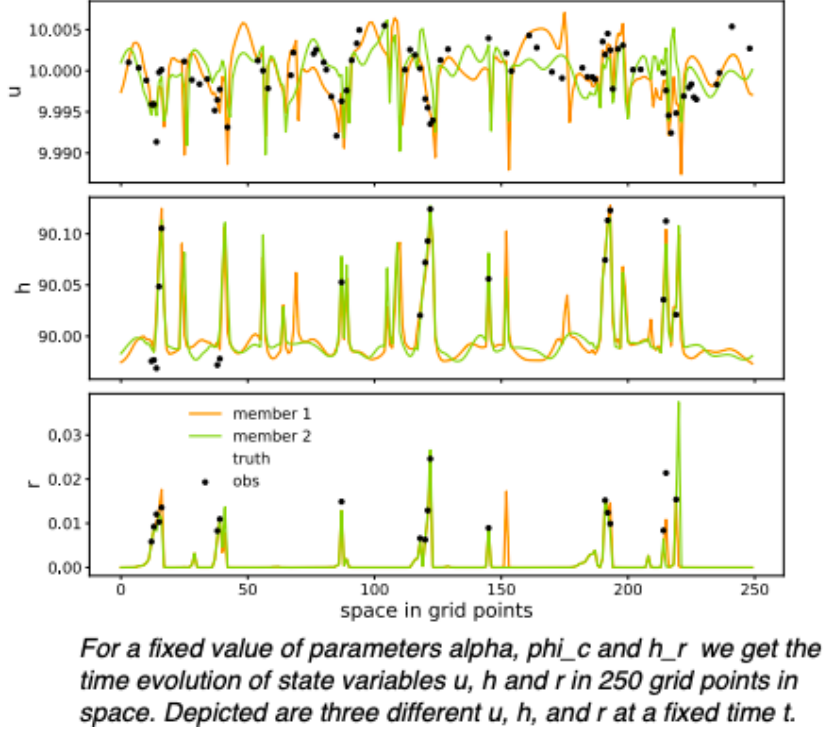


Figure 3: Feature selection interpreted as finding the most informative distances between data points.

Instead of selecting features based on how strongly they are correlated or what a neural network highlights, their method focuses on how the data is shaped—that is, which features best preserve the natural structure of the data. This is especially useful for our toy model (modified shallow water model), where every grid point gives us values like velocity (u), cloud height (h), and rain (r) over time, and space represents a feature. Since our goal is to predict the parameters (α , ϕ_c , h_r), this method helps us find which of those many features are actually most predictive.

The authors used their information imbalance method to examine how different national policies—such as lockdowns, testing, or school closures—affected the spread of COVID-19. They found that no single policy on its own was enough to make good predictions, but combinations of policies were much more effective. They also tested the method in materials modeling, selecting the most informative atomic descriptors from an initial set of 324. Using their approach, they found that around 16 features were sufficient to achieve nearly the same results, making the models faster and simpler without losing accuracy.

1 Introduction

The main benefits of these methods include model independence, the ability to handle nonlinear data and overlapping (redundant) features, and the capability to identify feature interactions and potential causal relationships. Limitations include the need for pairwise computations and ranking operations, which can be computationally intensive for large datasets.

In summary, the method offers a strong way to select important features in complex, high-dimensional data. For our project, applying this approach to our $u/h/r$ dataset should help us find a smaller set of meaningful features to predict the shallow water model's parameters. This can make our model simpler, easier to understand, and provide a valuable comparison against more traditional machine learning feature selection techniques.

2 Materials and Methods

2.1 Dataset

The dataset used in this project is the output of a modified shallow water model. This toy model produces the time evolution of three physical variables:

$$\mathbf{s}(t) = [u(t), h(t), r(t)]^T$$

where:

- u : velocity,
- h : cloud height,
- r : rain.

The model depends on three parameters:

$$\alpha, \quad \phi_c, \quad h_r$$

For each fixed parameter set (α, ϕ_c, h_r) , we obtain the time evolution of the state variables u , h , and r across 250 grid points in space.

Every spatial point at every time step is treated as a distinct feature. For example, a single simulation produces:

$$\text{features} = (\text{variable type}) \times (\text{grid points}) \times (\text{time steps})$$

The full dataset (`optimized_simulations_5k.nc`) contains 5000 such simulations. Our goal is to identify the most relevant features for predicting the parameters α, ϕ_c, h_r .

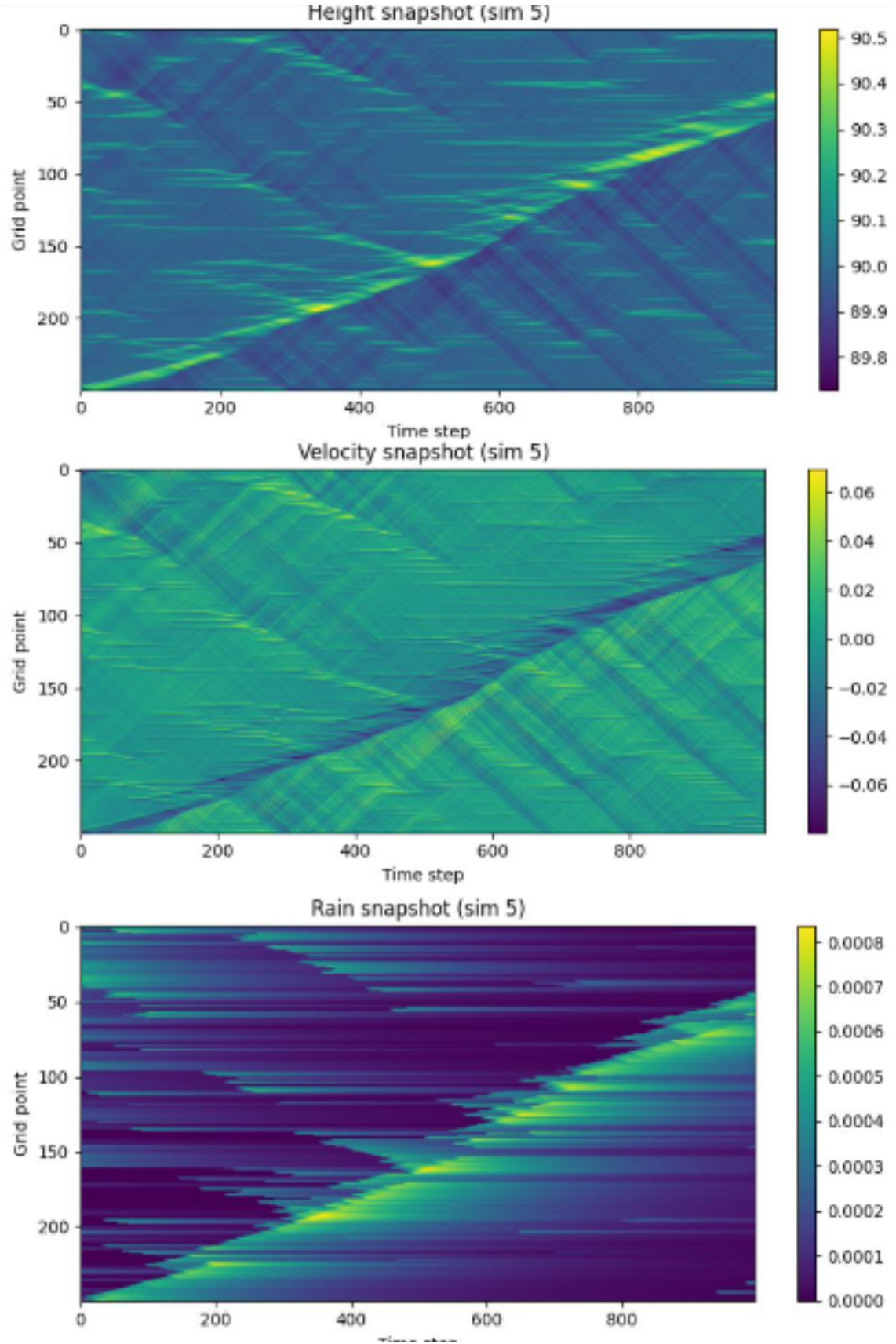


Figure 4: Raw simulation snapshots for velocity, height, and rain, illustrating the temporal and spatial evolution of each variable across the grid.

2.2 Feature Selection with Information Imbalance Method

We employed the *information imbalance* approach proposed by Glielmo et al., implemented in Python by Edoardo Cignoni [1]. This method determines how much predictive information one feature space contains about another.

Given two feature sets A and B representing the same n data points, the asymmetric information imbalance is computed as:

$$\Delta(A \rightarrow B) \in [0, 1]$$

where $\Delta = 0$ indicates that A fully explains B , and $\Delta = 1$ means A contains no predictive information about B . This asymmetry makes it possible to determine whether:

- A and B are equivalent,
- A contains B ,
- B contains A ,
- A and B are independent.

2.3 Code Implementation

The repository [1] contains the core implementation in `information_imbalance.py`, which provides:

1. Computation of pairwise distances (e.g., Euclidean) between data points in each feature space.
2. Copula-based transformation to normalize distances and remove effects of scale or non-uniform distributions.
3. Ranking of nearest neighbors in each space.
4. Computation of $\Delta(A \rightarrow B)$ and $\Delta(B \rightarrow A)$.

2 Materials and Methods

The principal function accepts two matrices A and B of size (n, m_A) and (n, m_B) , where rows are samples and columns are features. It outputs the directional imbalance scores, which can be used for asymmetric or symmetric comparisons.

We directly integrated this code into our workflow with minimal modifications, mainly adapting input preprocessing to our dataset.

```
def information_imbalance(A, B, metric="euclidean", mode="vectorized"):
    """
    Computes the information imbalance of going from a set of features
    A to a set of features B.

    Delta(A->B) = 2 <r^B | r^A=1> / N

    where r^B is the rank of B and r^A is the rank of A.
    This amounts to computing the rank r_ij^B for each pair i,j for
    which r_ij^A=1.
    The algorithm then scales as N^2.
    Arguments
    =====
    A      : ndarray, (num_samples, num_features_A) or (num_samples,)
             First set of features.
    B      : ndarray, (num_samples, num_features_B) or (num_samples,)
             Second set of features
    metric : str
             Metric used to compute the distances. Must be one of the allowed
             metrics in the `cdist` function of SciPy (the one used to actually
             compute distances).
             See: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cdist.html
    mode   : str
             How the Information Imbalance is computed.
             * 'vectorized' is faster but needs more memory. If it fails,
               it switches to the 'sequential' mode.
             * 'sequential' is slower, but should not run into problems when
               the amount of data is large.

    Returns
    =====
    information_imbalance : float
                           The information imbalance of going from A to B.

    References
    =====
    Glielmo et al., https://arxiv.org/abs/2104.15079
    """
```

Figure 5: Function for computing copula-transformed distances and directional information imbalance scores.

```
def feature_selection(
    features_dict, max_feats=5, metric="euclidean", mode="vectorized"
):
    """
    Performs a greedy feature selection as described in Glielmo et al. [1]
    The feature selection is based on the information imbalance criterion.
    Arguments
    =====
    features_dict : dict
        Initial set of features.
        Dictionary with key=feature_name and value=feature_values
    max_feats : int
        Maximum features to be selected
    metric : str
        Metric used to compute the distances. Must be one of the allowed
        metrics in the `cdist` function of SciPy (the one used to actually
        compute distances).
        See: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cdist.html
    mode : str
        How the Information Imbalance is computed.
        * 'vectorized' is faster but needs more memory. If it fails,
        it switches to the 'sequential' mode.
        * 'sequential' is slower, but should not run into problems when
        the amount of data is large.

    Returns
    =====
    selected_features : ndarray of str, (max_feats,)
        Names of the selected features
    imbalances : dict
        Dictionary storing the imbalances for each
        number of retained features. Structured as:
        imbalances  --- 0 --- 'A_B' : values
                        'B_A' : values
                        --- 1 --- 'A_B' : values
                        'B_A' : values
                        ...
    """
```

Figure 6: Workflow for computing feature selection based on information imbalance, illustrating the selection of top 5 features.

2.4 Workflow Overview

Our feature selection and comparison process consisted of the following steps for each parameter (α , ϕ_c , h_{rain}):

1. Feature selection with the Information Imbalance method:

- Due to the computational cost of working with the full dataset, we used a representative subset of simulations.

- The information imbalance method was applied to identify the **top 5 most informative features** for predicting each parameter.
- Our initial assumption was that each parameter is primarily related to a specific variable: α with velocity (u), ϕ_c with height (h), and h_{rain} with rain (r).

2. Model training and comparison:

- A machine learning model was trained to estimate each parameter using:
 - a) All features.
 - b) Only the top 5 features selected by the information imbalance method.
- The results were compared to assess the performance drop (if any) when using a reduced feature set.

3. Feature selection with the backpropagation method (Neural Network):

- A neural network was used to estimate feature importance via backpropagation.
- The top 5 features for each parameter were extracted using this method.

4. Final comparison:

- The same machine learning model was trained using:
 - a) All features.
 - b) Top 5 features from the information imbalance method.
 - c) Top 5 features from the backpropagation method.
- The results were compared to determine which approach — information imbalance, backpropagation, or no selection — offered the best trade-off between performance and complexity.

3 Results

In this section, we present the outcomes of our feature selection experiments using the *Information Imbalance* method proposed by Glielmo et al. (2023) and a neural network (NN) backpropagation-based method. We evaluate each approach by comparing the predictive performance of a Random Forest model trained on:

1. The full feature set,
2. The top 5 features selected by the Information Imbalance method,
3. The top 5 features selected by the NN method.

Our goal is to determine whether extreme dimensionality reduction can maintain acceptable predictive accuracy while improving interpretability.

3.1 Feature Selection with the Information Imbalance Method

Using the GitHub implementation of the Information Imbalance method, we identified the five most informative features for predicting each parameter (ϕ_c , h_{rain} , and α). The selected features are listed in Figure 7.

phic	h_rain	alpha
"rain_g200_t8"	"rain_g130_t0"	"rain_g190_t40"
"rain_g175_t64"	"rain_g230_t528"	"rain_g190_t192"
"rain_g130_t0"	"rain_g45_t32"	"rain_g190_t32",
"rain_g185_t208"	"rain_g25_t544"	"rain_g190_t184"
"rain_g180_t8"	"rain_g130_t8"	"rain_g190_t0"

Figure 7: Top 5 features selected for each parameter using the Information Imbalance method.

3.2 Comparing Top Features vs. All Features

To assess the trade-off between model complexity and performance, we trained Random Forest models using either all features or only the 5 features selected by the Information Imbalance method.

Parameter ϕ_c : This parameter represents the characteristic height and is predicted using rain-related features.

Parameter: ϕ_c (phic)

Metric	All Features	Glielmo Features	Relative Change
MAE	0.0149	0.0228	+53.1% $\uparrow$
R²	0.890	0.727	-18.3% $\downarrow$

Figure 8: Performance comparison for ϕ_c : All features vs. Information Imbalance selected features.

Parameter h_{rain} : This parameter corresponds to rainfall height, with early-time rain features being most influential.

Parameter: h_{rain}

Metric	All Features	Glielmo Features	Relative Change
MAE	0.0103	0.0143	+39.3% $\uparrow$
R²	0.804	0.608	-24.4% $\downarrow$

Figure 9: Performance comparison for h_{rain} : All features vs. Information Imbalance selected features.

Parameter α : This parameter relates to velocity. For this parameter, log-transformation was applied to improve prediction accuracy.

Parameter: α (alpha)

Metric	All Features	Glielmo Features	Relative Change
MAE ($\times 10^5$)	2.78	2.43	-12.7% ↓
R ²	0.964	0.953	-1.1% ↓

Figure 10: Performance comparison for α : All features vs. Information Imbalance selected features.

For α , the Glielmo method achieved slightly better MAE than the full feature set, while for ϕ_c and h_{rain} the reduced feature set retained between 60–73% of the full model’s predictive power despite using only 0.013% of the original features.

3.3 Feature Selection with Neural Network Backpropagation

We also applied a neural network-based approach, extracting the absolute mean weights from the first layer to rank features. The top 5 features for each parameter are shown in Figure 11.

phic	h_rain	alpha
"rain_g125_t944"	"velocity_g185_t688"	"height_g155_t560"
"rain_g135_t992"	"velocity_g155_t32"	"height_g145_t448"
"velocity_g175_t88"	"velocity_g185_t32"	"height_g205_t224"
"rain_g115_t960"	"velocity_g120_t0"	"height_g50_t696"
"velocity_g125_t0"	"velocity_g50_t784"	"height_g45_t744"

Figure 11: Top 5 features selected for each parameter using the neural network method.

These features differ substantially from those obtained via the Information Imbalance method, with the NN often prioritizing velocity and height features over rain features.

3.4 Summary of Observations

Across all parameters, the Information Imbalance method demonstrated a superior performance-to-complexity ratio compared to the NN approach. In particular:

- For α , the Glielmo method reduced MAE by 13% compared to the full set while using 5 features instead of 37,500.
- For ϕ_c and h_{rain} , the method retained substantial predictive power despite extreme dimensionality reduction.
- The NN method showed inconsistent performance, especially for α , where it led to catastrophic prediction failure.

4 Conclusion

The objective of this work was to compare two feature selection methods—*Information Imbalance* (Glielmo et al., 2023) and a neural network (NN) backpropagation-based approach—against the baseline of using the full feature set from the modified shallow water model dataset. The comparison focused on predictive accuracy, dimensionality reduction, and interpretability.

4.1 Overall Findings

- **Information Imbalance Method:** Achieved the best performance-to-complexity trade-off. In most cases, predictive accuracy remained close to that of the full feature set despite using only 5 features (0.0053% of the total). Notably, for α prediction, it outperformed the full feature set in MAE while retaining 95% of the R^2 score.
- **Neural Network Method:** Produced less consistent results. For ϕ_c and h_{rain} , accuracy was lower than both the full set and Glielmo method. For α , the method performed poorly, with a 92% drop in R^2 .
- **Full Feature Set:** Delivered the highest accuracy in some cases, but at the cost of enormous computational requirements and lack of interpretability.

4.2 Recommendations

- For applications where **interpretability** and **computational efficiency** are critical, the Information Imbalance method is the preferred choice.
- When **maximum accuracy** is the sole priority and resources allow, using the

full feature set can be beneficial, but requires careful regularization to prevent overfitting.

- The NN approach may benefit from architectural tuning or feature engineering, but in its current form, it is not recommended—especially for α prediction.

4.3 Comparison Figures

The following figures summarize the comparative performance of all three approaches across the three target parameters.

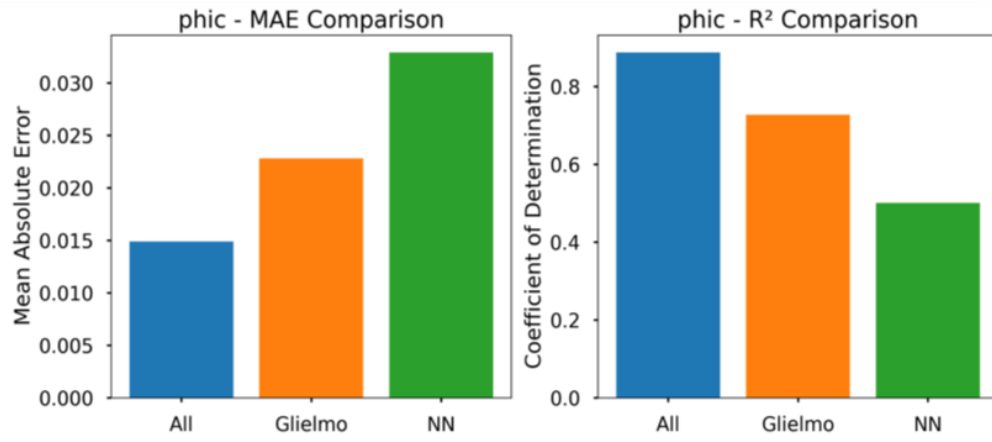


Figure 12: Performance comparison for ϕ_c : All features, Glielmo method, and NN method.

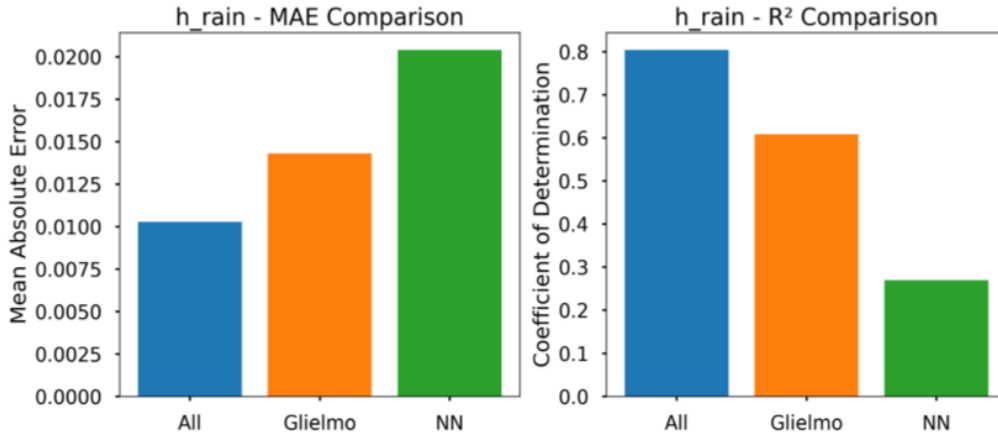


Figure 13: Performance comparison for h_{rain} : All features, Glielmo method, and NN method.

4 Conclusion

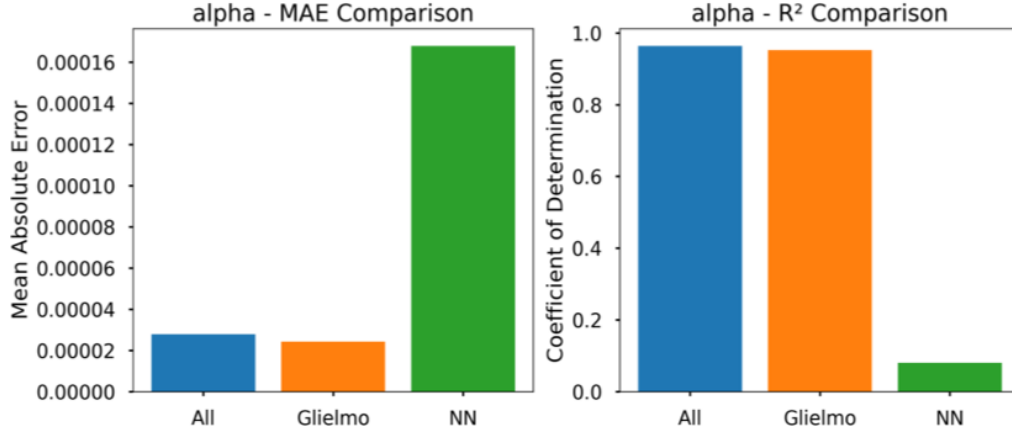


Figure 14: Performance comparison for α : All features, Glielmo method, and NN method.

In summary, the *Information Imbalance* method provides a highly effective means of identifying compact and informative feature subsets, enabling large reductions in dimensionality without sacrificing much predictive power. This makes it particularly valuable in scientific applications where both interpretability and computational feasibility are important.

Bibliography

- [1] E. Cignoni. *Information Imbalance GitHub Repository*. URL: https://github.com/ecignoni/Information_Imbalance (visited on 08/14/2025).
- [2] Aldo Glielmo, Claudio Zeni, Bingqing Cheng, Gábor Csányi, and Alessandro Laio. “Ranking the Information Content of Distance Measures”. In: *PNAS Nexus* (2022). Preprint arXiv:2104.15079v2 [stat.ML], 25 May 2022, pp. 1–9. DOI: 10.1093/pnasnexus/pgac039.